CardioCog-RAG

The complete study guide: every part of the project, explained from zero

What this is: One chapter per part of the project. Each chapter has the same shape: the idea in one line, what was done and how, the settings we chose and why, the result, and a sentence you can say when presenting.

Contents

1	The big picture	9	Explaining the model (SHAP)
2	Words you need	10	PET numbers
3	The data	11	Checking the model properly
4	The project folder	12	Coping with missing information
5	Cleaning and features	13	The brain scan images
6	Who we study, what we predict, the fair test	14	The evidence chatbot (RAG)
7	The five models and the score	15	Checking our own work
8	Which information matters; does history help	16	The doctor's screen
		17	Limits, the presentation, and questions

Every number in this guide was checked against the result files in `reports/`. Nothing here is invented. Research prototype, not for clinical use.

1 The big picture

In one line: We estimate, for a person without dementia today, the chance of a dementia diagnosis within 3 years, and we find out which kinds of information make that estimate good.

The problem. Dementia is the loss of memory and thinking to the point where a person cannot manage daily life. It usually comes on slowly over years. If we could tell early who is likely to develop it, doctors could watch those people more closely and plan ahead.

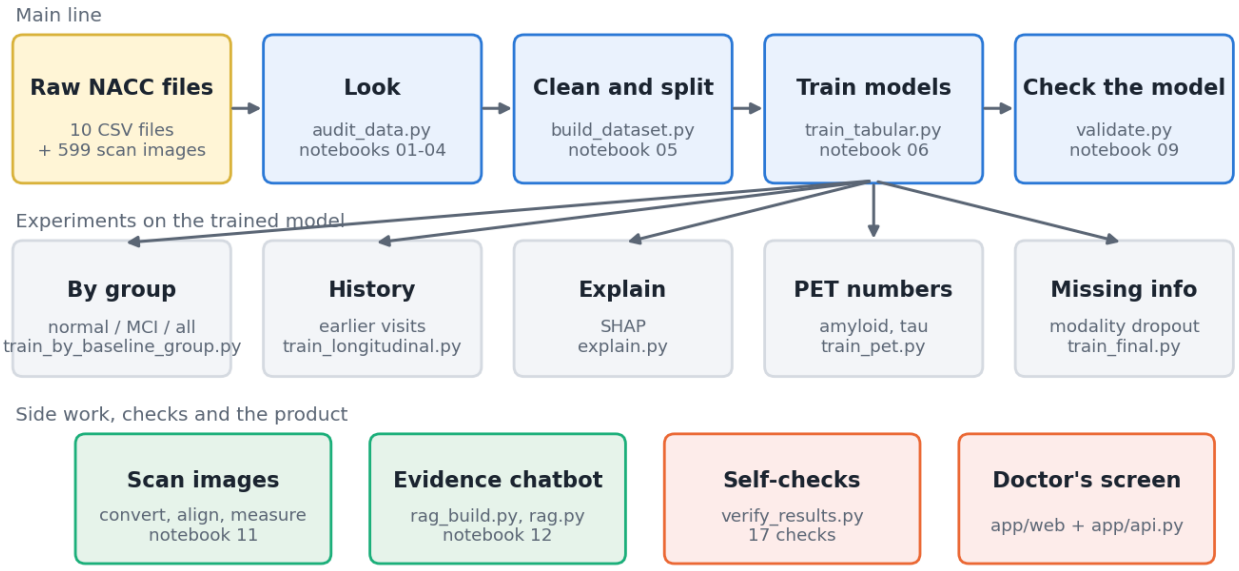
What we built. A computer program (a *model*) that looks at a person's information from one clinic visit and gives a number: the estimated chance of a dementia diagnosis in the next 3 years.

What we wanted to learn. Three questions:

- Can this be predicted at all, and how well?
- Which kind of information matters most? In particular, does **heart and blood-vessel health** help? That was the original idea behind the name 'CardioCog'.
- Do complicated modern AI models do better than simple ones?

The answers, in short.

- Yes: the score is 0.94 on a scale where 0.5 is a coin toss and 1.0 is perfect.
- What matters is how the person's brain is doing **today** (thinking tests and the doctor's rating). Heart and blood-vessel information added very little once those were known.
- No: a simple model and four more advanced ones score the same.



The whole project on one page. The top row is the main line; everything else builds on the trained model.

Say it like this: "Our project asks whether we can estimate a person's risk of dementia within three years, and which kinds of information actually help."

Careful: It is a research study on past data. It does not diagnose anyone and does not suggest treatment. Say this early.

2 Words you need

You do not need medical training. You need these words.

Medical words

Word	Plain meaning
Dementia	Memory and thinking loss severe enough that the person cannot manage daily life alone
Alzheimer's disease	The most common cause of dementia
Cognitive	To do with thinking and memory. 'Cognitive tests' are thinking tests
Cognitively normal	No thinking problem found at the visit
MCI	Mild cognitive impairment: a noticeable thinking problem, but the person still manages daily life. A halfway stage
Conversion	Moving from 'no dementia' to 'dementia'. A 'converter' is someone who did
CDR	Clinical Dementia Rating: a doctor's rating of how much daily life is affected. We use the total, 'CDR Sum of Boxes', from 0 (no problems) to 18
Cardiovascular	To do with the heart and blood vessels: blood pressure, diabetes, stroke, heart disease
APOE	A gene. One version, called e4, raises the risk of Alzheimer's disease. A person has 0, 1 or 2 copies
Amyloid, tau	Two proteins that build up in the brain in Alzheimer's disease
PET scan	A brain scan. A tracer is injected and the scan shows where it collects, which tells how much amyloid or tau is there

AI words

Word	Plain meaning
Model	A program that learns a pattern from examples and then applies it to new cases
Feature	One number the model is given about a person, such as age or a memory score. We use 48
Training	Showing the model people whose outcome we already know, so that it learns the pattern
Test set	People hidden during training and used once at the end, like an exam the student has never seen
AUROC	Our main score. Take one person who developed dementia and one who did not. AUROC is how often the model gives the higher risk to the right person. 0.5 = coin toss, 1.0 = perfect
Calibration	Whether the risk numbers are believable: when the model says 40%, do about 40 in 100 such people really develop dementia?
Setting (parameter)	A choice we fix before training, such as 'predict 3 years ahead' or 'use 500 trees'
Seed	A fixed number that makes the random parts of a program repeat exactly, so anyone can reproduce our results
Leakage	When the model accidentally gets to see the answer or the future. It makes results look better than they really are

Word	Plain meaning
z-score	How far a test score is from what is expected for a healthy person. 0 = typical, negative = worse
SHAP	A method that shows how much each feature pushed a prediction up or down
RAG	Retrieval-augmented generation: a chatbot that first looks up trusted documents and answers only from them

3 The data

In one line: One big table of clinic visits gives us 48 features and the outcome; four small scan tables give us 4 more; the other five files are not used.

Where it comes from. NACC (National Alzheimer's Coordinating Center) collects data from 46 research centres in the United States. Volunteers come in about once a year and the same information is recorded at every visit. We have the June 2026 release, called Freeze 74.

How much. 57,038 people and 217,598 visits, from 2005 to 2026. A person has between 1 and 21 visits.

The files

File	What it holds	Used for
`investigator_ftldlbd_nacc74.csv`	The main file. One row per visit, 2,649 columns: age, illnesses, blood pressure, medicines, thinking tests, the doctor's rating, the diagnosis, the APOE gene	All 48 main features and the outcome
Two amyloid summary files (`...amyloidpetgaain...`)	One row per amyloid scan: the amount of amyloid (Centiloids) and positive / negative	2 PET features
Two tau files (`...taupetnpdka...`)	One row per tau scan: the amount of tau in each brain region	2 PET features
Two regional amyloid files, the FDG file, the PET quality list, the CLARiTI tracking file	Region-by-region values, another scan type, and bookkeeping	Not used
`data/raw/pet/`	599 raw scan images, about 100 GB	The image experiment (chapter 13)

The five kinds of information

Kind	What it is	Examples of real column names	Features
Demographics	Basic facts about the person	`NACCAGE`, `NACCSEX`, `EDUC`	8
Cardiovascular	Heart and blood-vessel health	`HYPERTEN`, `DIABETES`, `CBSTROKE`, `BPSYS`	25
Cognitive	Thinking tests, mood, and the doctor's rating	`MOCATOTS`, `CRAFTDVR`, `CDRSUM`, `NACCUDSD`	12
Genetics	The APOE gene and family history	`NACCNE4S`, `NACCFAM`	3
PET	Amount of amyloid and tau	`CENTILOIDS`, `METATEMPORALSUVR`	4

The full list, with every column, is in `docs/features_by_category.md`.

Example: How 2,649 columns became 48 features: 1,862 columns are almost empty (forms for rare conditions) and were dropped. Of the 787 left, 76 belong to our five kinds of information. Those 76 are combined into 48, for example 20 separate tests become 6 scores.

Privacy. The data are covered by an agreement with NACC. The `data/` folder is never uploaded. Reports contain only totals and averages, never a single person's values. A safety check in the code stops any

report that contains a participant ID.

> **Say it like this:** "We used the NACC data set: fifty-seven thousand volunteers, seen about once a year, with five kinds of information on each."

4 The project folder

In one line: Scripts do the work, reports hold the results, notebooks tell the story, and data and models stay private on the laptop.

Folder	What it holds	Open it when
`data/raw/`	The original NACC files. Only ever read, never changed	Never during a presentation: file names contain participant IDs
`data/processed/`	What our scripts compute: the cleaned table, the splits, the converted scan images	You are re-running something
`scripts/`	22 Python programs. Each does one step	Someone asks 'show me the code'
`notebooks/`	12 notebooks, numbered in order, mixing explanation, code and charts	You want to walk through the story
`reports/`	Results written by the scripts: numbers (.json), summaries (.md), charts (`figures/`)	Someone asks 'where does that number come from?'
`models/`	The trained model used by the screen	Never opened by hand
`app/`	The doctor's screen: `web/` is the page, `api.py` runs the model	The live demo
`rag/`	The list of research papers and the test questions for the chatbot	Explaining the chatbot
`docs/`	Guides for people, including this one	Preparing

The scripts, grouped

Group	Scripts	Chapter
Shared tools	`nacc_utils.py` (reading NACC files, code meanings), `modeling.py` (the five models, the scoring)	5, 7
Look	`audit_data.py`	3
Prepare	`build_dataset.py`	5, 6
Train and test	`train_tabular.py`, `train_by_baseline_group.py`, `train_longitudinal.py`	7, 8
Understand	`explain.py`, `train_pet.py`, `validate.py`, `train_final.py`	9 to 12
Scan images	`pet_convert.py`, `pet_register.py`, `pet_suvr.py`, `train_pet_image.py`, `view_pet.py`	13
Chatbot	`rag_build.py`, `rag.py`, `rag_eval.py`	14
Self-checks	`verify_results.py`, `check_clinical_behaviour.py`	15
Screen	`inference.py`, `app/api.py`, `app/web/`	16

The notebooks

Notebook	What it shows
01 Data catalogue	Every file, how full each column is, what the missing-value codes mean
02 Longitudinal and outcomes	How often people come back, and how diagnoses change over time
03 Feature categories	The five kinds of information, one by one
04 PET	Who has scans, what the scan numbers look like
05 Cleaning and cohort	Cleaning, who is studied, the outcome, the split
06 Models	The five models, which information matters, whether history helps
07 Explainability	SHAP
08 PET numbers	Do amyloid and tau add anything?
09 Validation	Subgroups, believable risks, a second method
10 Missing modalities	Coping when information is missing
11 PET images	From raw scanner files to measurements and an image model
12 Evidence assistant	The chatbot and how it was tested

5 Cleaning the data and building features

In one line: Raw columns are full of special codes and scores that cannot be compared directly, so we clean them and turn them into 48 fair, comparable numbers.

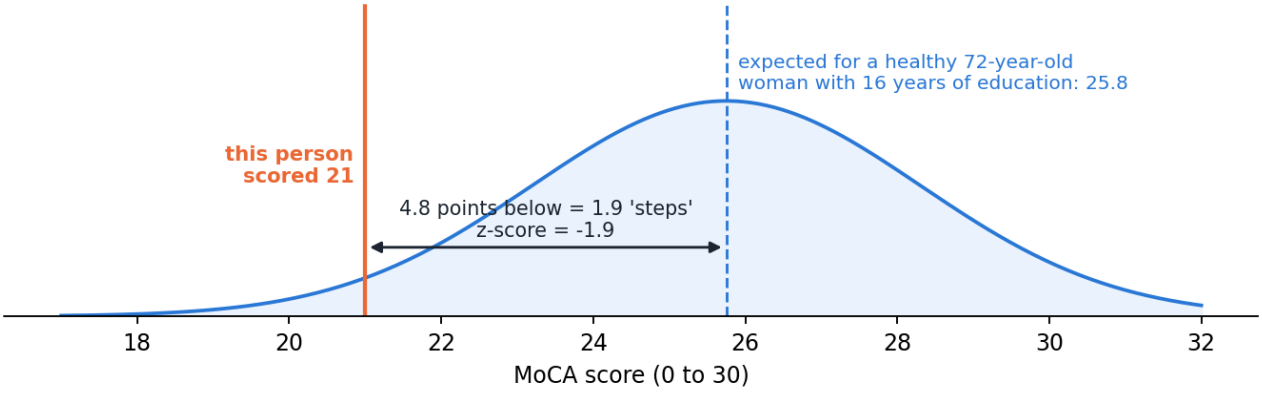
Script	`build_dataset.py (first half)`
Reads	The main clinical file and the four PET tables
What happens inside	1. Turns 'no answer' codes into missing. 2. Turns illnesses into yes/no and carries a 'yes' forward to later visits. 3. Turns every test score into a z-score and averages them into six area scores. 4. Counts behaviour symptoms and failed tests. 5. Attaches each person's first passed PET scan to a visit on or after the scan.
Writes	`data/processed/visits.parquet` (the cleaned table, one row per visit)

Problem 1: special codes

NACC uses numbers to mean 'no answer', and the meaning changes by column. 88 is a real age and a real blood pressure, but in 'years smoked' it means 'never smoked'. If we treated every 88 as missing we would delete every 88-year-old. So the correct codes were written down column by column (`reports/variable_map.md`).

Problem 2: test scores are not comparable

A score of 21 on a thinking test means something different for a 60-year-old professor and a 90-year-old who left school at 12. Also, the centres changed their tests in 2015. Our solution is the **z-score**: compare each score with what a healthy person of the same age, sex and education scores.



Real numbers from our data. Expected score = 23.0 - 0.059 x age + 0.33 (if female) + 0.42 x years of education, learned from 6,264 healthy people.

Example: A 72-year-old woman with 16 years of education scores 21 on the MoCA. Healthy people like her score 25.8 on average, give or take 2.6. She is 4.8 points below, which is 4.8 / 2.6 = 1.9 steps. Her z-score is **-1.9**: clearly below expected.

The z-scores of the tests in one area are then averaged, so each person gets six scores: screening, memory, attention, speed and planning, language, drawing. A person only has the tests their visit included, and the score is built from those.

Problem 3: the healthy reference must not peek

The 'healthy people' used as the reference are cognitively normal people **from the training group only**. If test-group people helped define 'normal', a little information about them would leak into the model.

Problem 4: facts that do not change

- If someone has ever had a stroke, the answer stays 'yes' at later visits.
- Blood pressure and weight use the latest measured value up to the visit.
- Nothing is ever taken from a **later** visit. An early version did this for education; our checks caught it and we fixed it (chapter 15).

> **Say it like this:** "We cleaned the data and expressed every test score relative to healthy people of the same age, sex and education, so old and new tests share one scale."

6 Who we study, what we predict, and the fair test

In one line: We take each person's first visit, ask whether dementia was diagnosed within 3 years, and test on people and whole hospitals the model never saw.

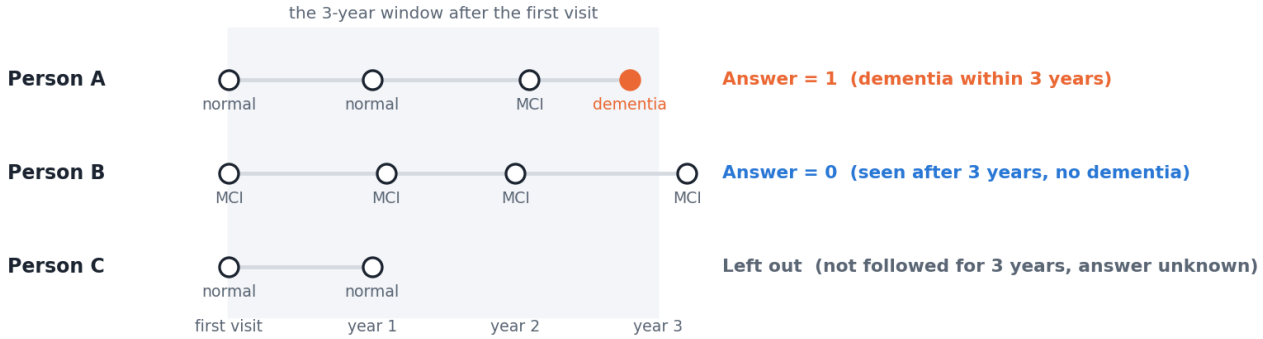
Script	`build_dataset.py (second half)`
Reads	The cleaned visits table
What happens inside	1. Marks who is eligible. 2. Works out the yes/no answer for each visit. 3. Splits people into four groups.
Writes	The split for every person, and `reports/phase2_cohort_summary.json`

Who is included

Step	People left
Everyone in the data	57,038
First visit was in person, on form versions 1 to 3 (these have the thinking tests)	55,461
No dementia at that first visit	38,107
We can tell what happened within 3 years	19,470
...of whom developed dementia	2,818 (14.5%)

About half of the eligible people drop out at the fourth step. They were not followed for 3 years, so we simply do not know their answer.

What is predicted



How the yes/no answer is decided. Only later visits are used for the answer; only the first visit is used for the features.

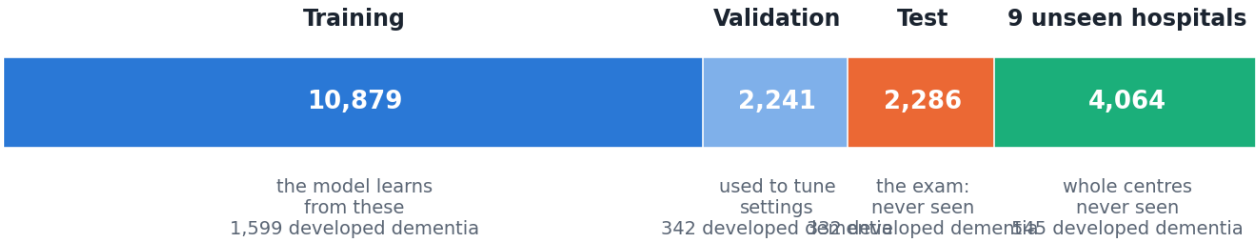
Three groups hide inside 'no dementia today'

At the first visit	People	Developed dementia in 3 years
Cognitively normal	12,377	198 (1.6%)
MCI	5,993	2,485 (41.5%)
Impaired, but not MCI	1,100	135 (12.3%)

Almost 9 in 10 of the people who developed dementia already had MCI. Remember this; it explains the results in chapter 7.

The fair test

A student who has seen the exam paper gets a meaningless mark. A model is the same. So people are split **before** any training:



The four groups. A person is in exactly one. Seed 2026 makes the split repeat exactly.

Setting	Value	Why
Prediction window	3 years	Long enough to matter, short enough that many people were followed that long
Split by	Person, never by visit	If one person's visits were on both sides, the model would recognise the person
Unseen hospitals	9 of 46 centres (20%), kept aside whole	The hardest test: does it still work at a hospital it has never seen?
The rest	70% training, 15% validation, 15% test	A common split; each part keeps the same mix of normal and MCI
Seed	2026	So the split can be reproduced exactly

Say it like this: "We studied nineteen thousand people without dementia, and tested on two thousand people and nine whole hospitals the model had never seen."

7 The five models and the score

In one line: Five very different models all score about 0.94, and that number needs to be read together with 0.84 (MCI) and 0.88 (normal).

Script	`train_tabular.py, train_by_baseline_group.py (models defined in modeling.py)`
Reads	The cleaned table and the split
What happens inside	Trains each model on the training group, tunes it on the validation group, then scores it once on the test group and once on the unseen hospitals.
Writes	`reports/phase3_tabular_results.json`, `reports/phase3_by_baseline_group.json`

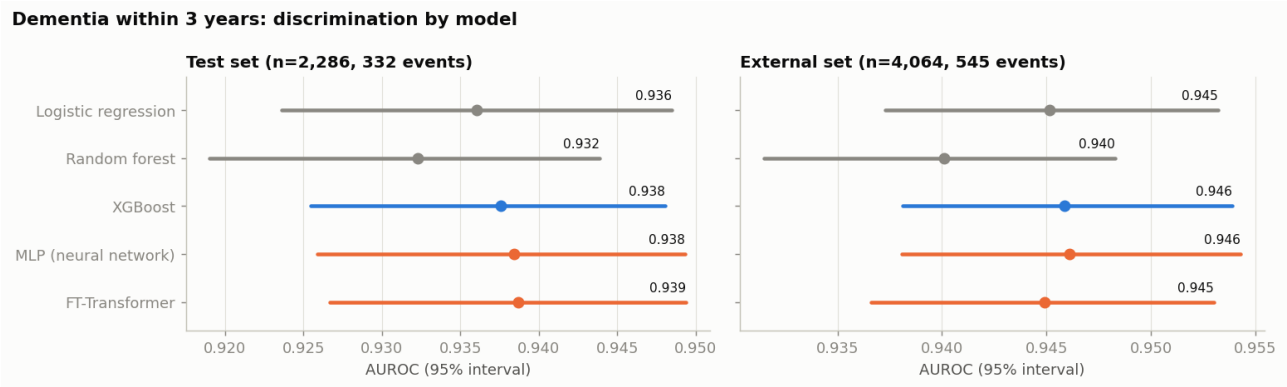
The five models, simplest first

Model	The idea in plain words	Main settings
Logistic regression	A weighted checklist: each feature gets a weight, and the weights are added up	Strength of simplification chosen from 5 values
Random forest	Many decision trees, each a chain of yes/no questions, that vote	500 trees
XGBoost	Trees built one after another, each fixing the mistakes of the ones before. Our main model	Up to 3,000 small trees; stops when the validation score stops improving; learning rate 0.03; tree depth 3, 4 or 6
Neural network (MLP)	Layers of simple units that combine the features in many ways	Two layers, 128 and 64 units
FT-Transformer	A modern deep-learning design related to the technology behind chatbots, adapted for tables	3 layers

All except logistic regression are trained three times with different seeds (0, 1, 2) and averaged, so one lucky run cannot fool us.

Results

Model	Test group	9 unseen hospitals
Logistic regression	0.936	0.945
Random forest	0.932	0.940
XGBoost	0.938	0.946
Neural network	0.938	0.946
FT-Transformer	0.939	0.945



The five models side by side. The differences are smaller than the uncertainty.

Finding 1: the advanced models did not beat the simple one. On table-like data of this size this is a common result.

Reading the 0.94 honestly

People with MCI convert 41.5% of the time, normal people 1.6%. So just knowing which group someone is in already separates them well: the status alone scores 0.84. The fair question is how well the model does **inside** each group.

Who	Test: people / cases	Score	Unseen hospitals: people / cases	Score
Everyone without dementia	2,286 / 332	0.937	4,064 / 545	0.946
Cognitively normal only	1,411 / 24	0.884	2,624 / 37	0.876
MCI only	732 / 289	0.835	1,291 / 496	0.846

Careful: The 0.88 for normal people rests on only 24 and 37 cases, so it could really be anywhere from about 0.82 to 0.94. And because dementia is rare in that group, even a good ranking gives many false alarms.

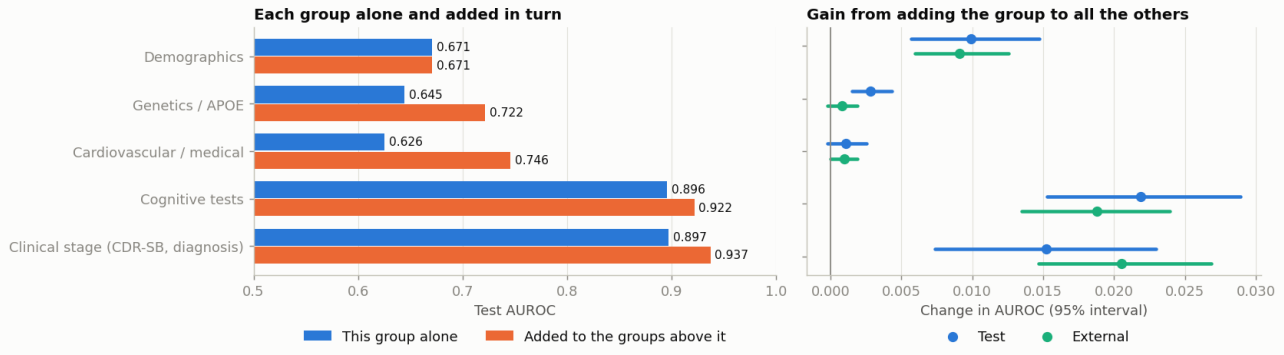
Say it like this: "All five models score about 0.94. But that mixes two very different groups. Among people who already have mild impairment, it is 0.84, which is the fairer number."

8 Which information matters, and does history help?

In one line: Thinking tests and the doctor's rating carry the prediction. Cardiovascular information and earlier visits each add very little in this data.

Experiment 1: remove one kind of information at a time

We train the model with everything, then again without one kind, and see how much the score drops. A big drop means that kind matters.



Left: each kind alone, and added in turn. Right: how much each kind adds when the model already has all the others.

Kind of information	Score using only this	What it adds to all the others
Thinking tests	0.90	+0.022
Doctor's rating and status	0.90	+0.015
Demographics	0.67	+0.010
Genetics (APOE)	0.64	+0.003
Cardiovascular	0.63	+0.001

Finding 2: cardiovascular information added about +0.001. On its own it does carry some signal (0.63), but once the model knows how the person is thinking, it has little left to add.

Why might that be?

- People join the study at about age 71. Damage from high blood pressure mostly happens in middle age, which we cannot see.
- Any damage already done is probably already showing up in the thinking-test scores.
- Our cardiovascular information is mostly simple yes/no history and one blood-pressure reading per visit.

Careful: Never say cardiovascular health is 'useless' or 'unimportant for the brain'. The safe sentence: 'In this NACC group and this 3-year setup, cardiovascular features added little beyond current cognitive status.'

How much do the doctor's rating and status drive the score?

Model uses	Everyone	Normal only	MCI only
Status only	0.84	0.50	0.50
CDR rating and status	0.90	0.61	0.75

Model uses	Everyone	Normal only	MCI only
Everything except CDR and status	0.92	0.87	0.81
Everything except CDR, status and thinking tests	0.75	0.78	0.64
Everything	0.94	0.88	0.84

Without any information about current thinking, the score falls from 0.94 to 0.75. This is not cheating, because all of it is recorded at the first visit and the outcome comes later. But it means the model mostly measures how far along a person already is.

Experiment 2: does a person's history help?

Script: `train_longitudinal.py`. For people with at least two earlier visits, we compare using only today's visit with also using the past. Two ways: simple 'how fast is each score changing' numbers, and a **GRU**, a model that reads up to 6 visits in order.

Method	Test group	Unseen hospitals
Today's visit only	0.919	0.953
+ rate-of-change numbers	0.922 (+0.003)	0.956 (+0.003)
GRU reading the visits in order	0.920 (+0.000)	0.958 (+0.005)

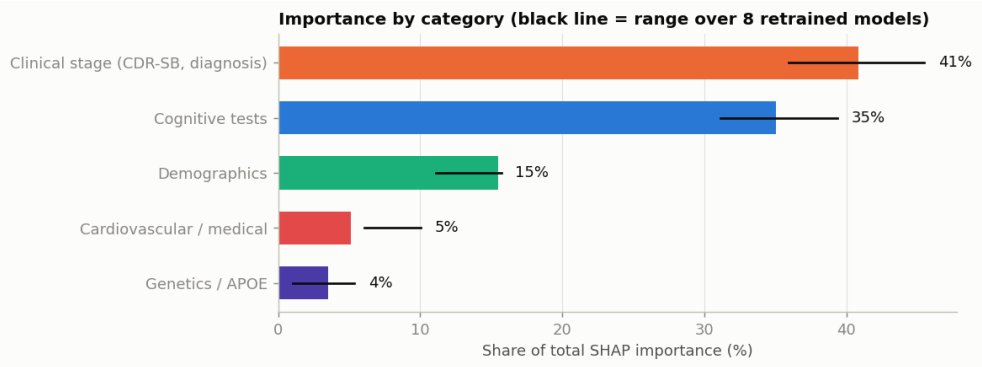
Finding 3: history added only +0.000 to +0.005. If someone has been getting worse, today's scores are already low. Knowing how they got there adds little once you know where they are.

> **Say it like this:** "Once we know how a person is thinking today, their cardiovascular record and their past visits add very little. We report that as a finding about this data, not as proof that they do not matter."

9 Explaining the model (SHAP)

In one line: SHAP shows what the model leans on: about three quarters of every prediction comes from the doctor's rating, status and thinking tests.

Script	`explain.py`
Reads	The trained XGBoost model and the test group
What happens inside	For every test person, SHAP splits the prediction into one contribution per feature. We average these, add them up per kind of information, and retrain 8 times on reshuffled data to check the ranking does not change.
Writes	`reports/phase7_shap_results.json`



Share of the model's reasoning that comes from each kind of information.

Kind of information	Share of the prediction
Doctor's rating and status	about 41%
Thinking tests	about 35%
Demographics (mainly age)	about 15%
Cardiovascular	about 5%
Genetics	under 5%

The single most important features, in order: CDR Sum of Boxes, having MCI, the memory score, age, the speed-and-planning score.

How to read SHAP for one person: each feature gets a plus or minus. A low memory score pushes the risk up; a CDR of 0 pushes it down. The pluses and minuses add up to the final risk. This is what the bars on the doctor's screen show.

Careful: SHAP shows what the **model** used, not what **causes** dementia. Example: lower body weight raised the predicted risk. That is not because being thin causes dementia; people tend to lose weight in the years before a diagnosis.

Say it like this: "We can explain every prediction. About three quarters comes from how the person's brain is doing today."

10 PET numbers

In one line: Amyloid on its own is a strong warning sign, but adding the PET numbers to everything else did not clearly improve the prediction.

Script	`train_pet.py`
Reads	The main model's predictions and the four PET tables
What happens inside	Takes the main model's risk for each scanned person and combines it with the PET numbers using a very small second model (called late fusion). Compares the score with and without PET.
Writes	`reports/phase4_pet_results.json`

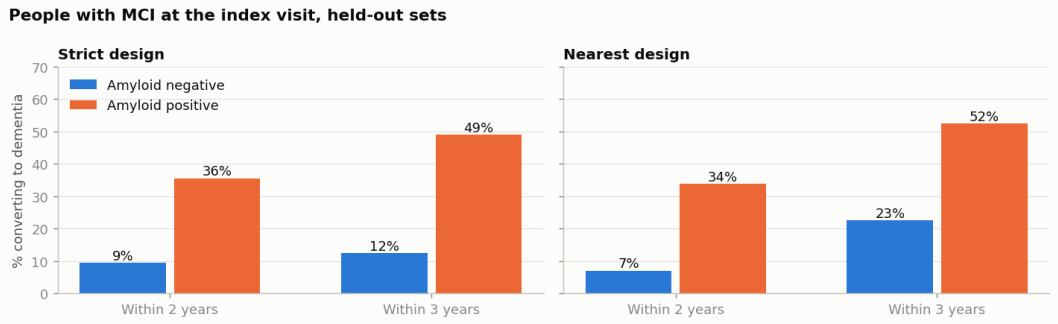
Who has PET numbers. About 5,100 people have amyloid numbers and 2,724 have tau. But most scans are recent, so few of these people have been followed for 3 years afterwards.

Why a small second model. With only a few dozen dementia cases among scanned people, a big model would just memorise them. So the big model learns from everyone, and only a tiny model with a handful of numbers is fitted on the scanned people.

Two ways of matching a scan to a visit, to be safe: the strict way (scan on or before the visit: clean, but few people) and the nearest way (closest visit: more people, but the scan is often a few weeks after the visit).

Results

- **On its own, amyloid matters.** Among people with MCI, about 34% of those who were amyloid positive developed dementia within 2 years, against about 7% of those who were negative.
- **Added to everything else, no clear gain.** Example at the unseen hospitals, strict matching, 3 years: 0.939 without amyloid, 0.946 with it, on 191 people and 27 cases. That difference is well inside the uncertainty.



Conversion to dementia among people with MCI, by amyloid status.

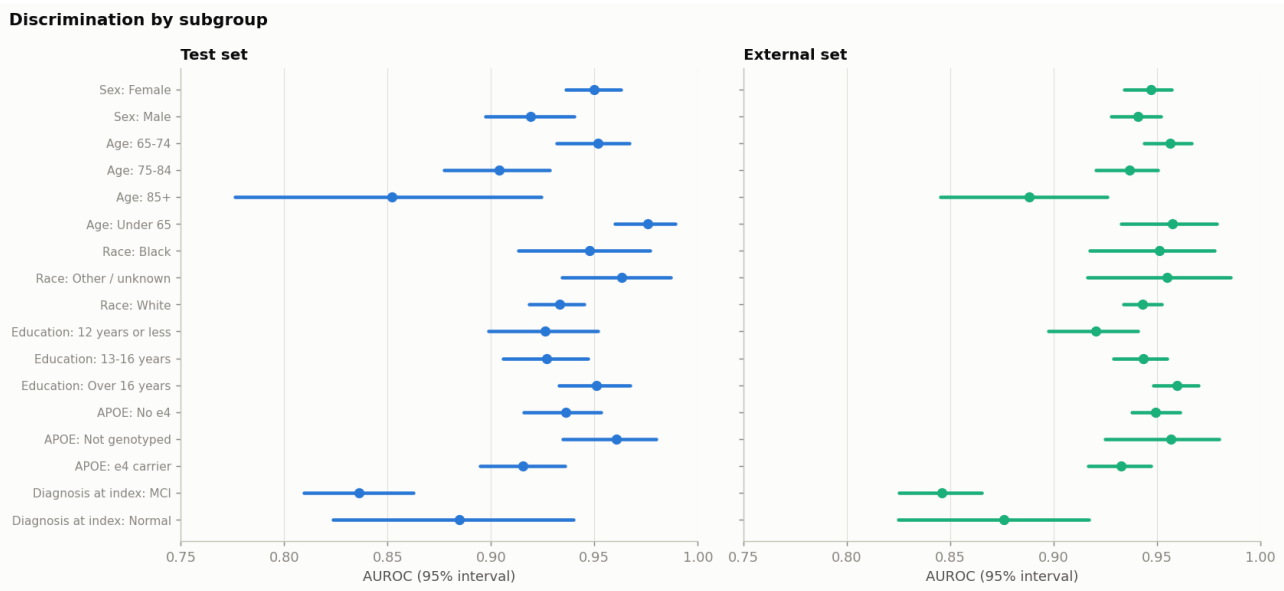
Careful: Say 'we could not show that PET improves the prediction', not 'PET does not help'. With so few followed-up cases, a real but modest benefit cannot be ruled out.

Say it like this: "Amyloid on its own is a strong marker, but once the model has the thinking tests, we could not show that PET adds to the prediction."

11 Checking the model properly

In one line: One score is not enough, so we checked the model in subgroups, checked that its risk numbers are believable, and repeated the analysis a second way.

Script	`validate.py`
Reads	The saved predictions for the test group and the unseen hospitals
What happens inside	1. Scores the model inside subgroups. 2. Tries several risk cut-offs. 3. Compares predicted with actual rates. 4. Asks whether using the model would beat simple rules. 5. Repeats the analysis with a method that includes everyone.
Writes	`reports/phase8_validation_results.json`



The score inside each subgroup, test group and unseen hospitals.

Subgroups. Similar across sex, race and education: 0.92 to 0.96 in the test group (0.95 for women, 0.92 for men). Weaker for people aged 85 and over: 0.85 in the test group, 0.89 at unseen hospitals.

Are the risk numbers believable? In the test group the average predicted risk and the actual rate are both 14.5%. At the unseen hospitals the model is slightly too high: 15.0% predicted against 13.4% actual.

What if we used a cut-off? If everyone with a predicted risk above 20% were flagged:

Measure	Value	Meaning
Flagged	22% of people	About 1 in 5 would be flagged
Sensitivity	83%	Of those who developed dementia, 83 in 100 were flagged
Specificity	88%	Of those who did not, 88 in 100 were correctly not flagged
Positive predictive value	55%	Of those flagged, 55 in 100 really developed dementia

A second method. Our yes/no question leaves out people not followed for 3 years. A different method (survival analysis) can include them. It led to the same conclusions, including that cardiovascular history added nothing measurable.

Say it like this: "The model performs similarly across sex, race and education, is weaker in the oldest people, and its risk numbers match what actually happened."

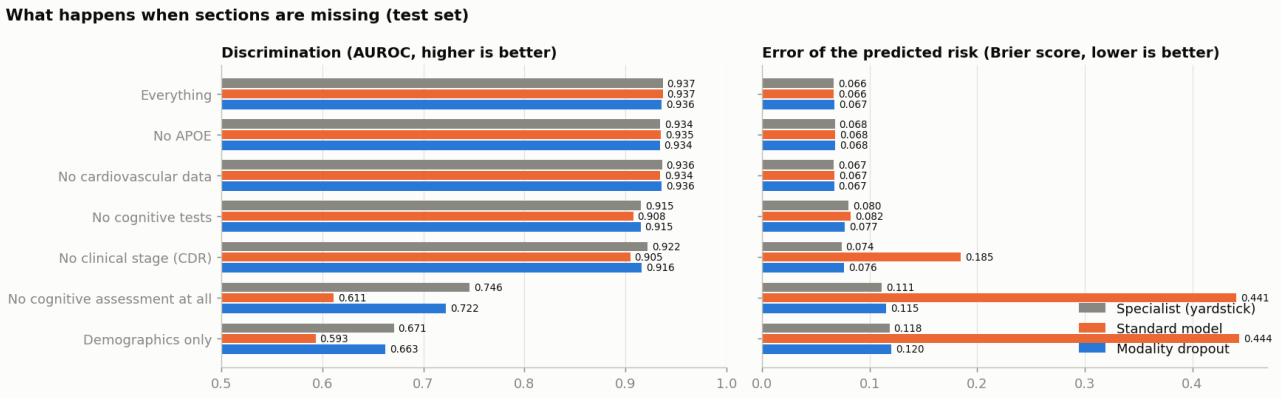
12 Coping with missing information

In one line: A normal model gives nonsense when a whole section is missing; training it with sections hidden at random fixes that.

Script	`train_final.py`
Reads	The cleaned table and the split
What happens inside	Makes 4 extra copies of the training data, and in each copy blanks out whole sections at random. Trains one model on all of it. Compares it with a normal model and with models built specially for each situation. Saves the result for the screen.
Writes	`reports/phase6_fusion_results.json` and the model in `models/`

The problem. In a real clinic not every patient has had every test. A model trained only on complete records has never seen 'no thinking tests at all' and does not know what to do.

The idea: modality dropout. During training we hide whole sections (genetics, cardiovascular, thinking tests, doctor's rating), each with a 35% chance, again and again. The model practises every combination of gaps.



Left: the score. Right: the error of the risk numbers (lower is better). Grey = a model built specially for that situation, the best possible.

Situation: no thinking tests, no CDR, no status	Error (lower is better)
Normal model	0.44 (worse than always guessing the average, which gives 0.12)
Our model trained with hidden sections	0.12
A model built specially for this situation	0.11 (the best possible)

One model now handles any combination, almost as well as a specialist for each case. This is the one place where an idea from deep learning clearly helped.

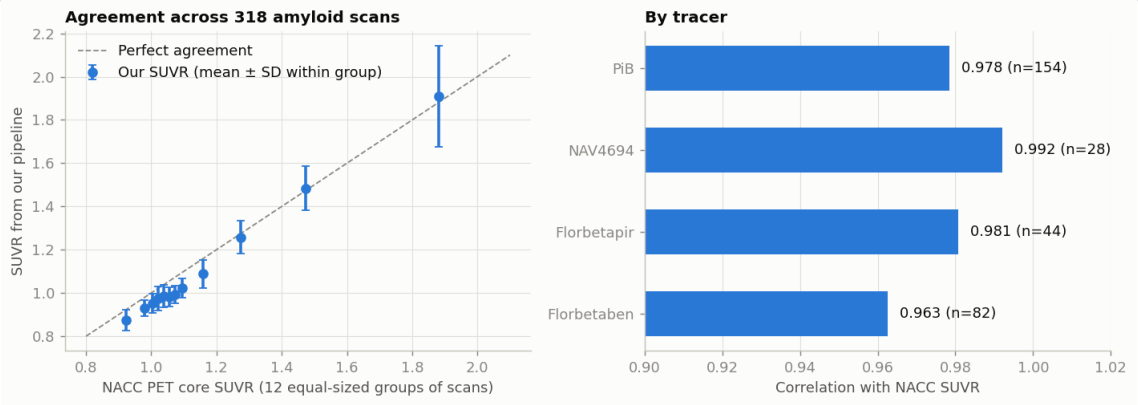
Say it like this: "Real patients do not have every test. We trained the model with whole sections hidden at random, so one model gives sensible estimates whatever is missing."

13 The brain scan images

In one line: We turned 599 raw scanner files into usable 3D pictures, proved our measurements match the experts', and trained an image model, but did not use images to predict dementia.

This is a separate experiment from the main model. It answers: can we handle real medical images correctly?

Step	Script	What happens
1. Convert	`pet_convert.py`	Each scan arrives as hundreds or thousands of small files from the scanner. We combine them into one 3D picture per scan
2. Line up	`pet_register.py`	Every head is in a different position and size, from about 20 scanner models. We rotate, shift and scale each picture so the same spot is the same part of the brain in all of them. This is called registration
3. Measure	`pet_suvr.py`	From our pictures we compute SUVR: the signal in the outer brain divided by the signal in a reference area. NACC's experts had measured the same thing with a different method, so we compare
4. Image model	`train_pet_image.py`	A 3D CNN (a deep-learning model for 3D pictures) learns to say whether a scan is amyloid positive



Our measurement against the experts' measurement for the same scans.

Result	Value
Scans converted	599 (318 amyloid, 232 tau, 49 FDG)
Agreement of our amyloid measurement with NACC's	0.98 on 318 scans (1.0 would be identical)
Telling amyloid positive from negative: the one simple number (SUVR)	0.98
Telling amyloid positive from negative: our 3D CNN	0.96

Settings for the CNN: pictures resized to 80 x 96 x 80, 40 rounds of training, and tested 5 ways so that every scan is tested exactly once by a model that never saw it.

Finding 4: the simple number beat the deep model. We also tried Grad-CAM, a method meant to show where in the picture the model looks. The map was blurry and did not point at the right regions. We report that as it is.

What we did NOT do

We did not predict dementia from the images. Only **5 of the 446** people with images later developed dementia. Nothing can learn a pattern from 5 examples, so claiming otherwise would be dishonest.

> **Say it like this:** "We built a full image pipeline from raw scanner files and checked it against the experts' numbers: 0.98 agreement. We deliberately did not predict dementia from images, because only five people with images developed it."

14 The evidence chatbot (the RAG in the name)

In one line: A chatbot that answers only from a small library of research papers, shows its sources, and says so when it has no answer.

The problem with ordinary chatbots. They can sound confident and be wrong. In medicine that is dangerous.

The RAG idea. Retrieval-augmented generation means: look it up first, then answer only from what was found.

Step	Script	What happens
1. Build the library	`rag_build.py`	Downloads the free full text of the listed papers, cuts each into paragraphs, and stores a 'meaning fingerprint' of each paragraph
2. Search	`rag.py`	For a question, runs two searches: one by meaning and one by keywords, and merges the two rankings. If nothing is similar enough, it answers 'I could not find evidence for that'
3. Answer	`rag.py`	A small language model on the laptop writes an answer from the found paragraphs with numbered sources. If it fails to cite properly, the chatbot simply quotes the paragraphs
4. Guardrail	`rag.py`	Questions asking for personal advice ('what medicine should my father take?') get a notice that it cannot advise on a person, and only general published information
5. Test	`rag_eval.py`	52 test questions of four kinds

Settings and results

Item	Value
Library	11 papers, 447 paragraphs (12 were listed; one could not be downloaded)
Topics	Risk factors, blood pressure and diabetes, APOE, amyloid and tau PET, diagnostic criteria
Right paper found first	32 of 32 questions
Personal-advice questions flagged	8 of 8
Off-topic questions declined	6 of 6
Language model wrote a properly cited answer	about 1 time in 5; otherwise direct quotation
Privacy	Everything runs on the laptop. Nothing is sent to the internet

Careful: Be open about the limits. The library is small, and we wrote the test questions ourselves, so finding the right paper every time was an easy test. The small language model is weak, which is why quoting is the fallback.

Say it like this: "The chatbot answers only from a small library of research papers and shows its sources. If the library has no answer, it says so, and it does not give personal medical advice."

15 Checking our own work

In one line: A separate program re-checks the data, the answers and the scores in 17 independent ways. It found two real mistakes, which we fixed.

Script	`verify_results.py`
Reads	The raw files, the cleaned table, the saved predictions and the reports
What happens inside	Recomputes things a second, independent way and tests properties that must be true if the work is sound.
Writes	`reports/verification_report.json`

What is checked	How	Result
The raw files were never changed	File sizes and dates against the first audit	Pass
Nobody is in two groups; unseen hospitals really are unseen	Counts overlaps	0 people, 0 centres shared
No information from the future	Deletes every later visit, rebuilds the features, checks nothing changed	Identical for 1,500 people x 67 features
The yes/no answers are right	Recomputes them with a separate, simpler program	0 disagreements in 11,403 visits
The model is not cheating	Shuffles the answers and retrains; the score must fall to a coin toss	0.51
The reported scores are right	Recomputes them from the saved predictions	Exact match
The result repeats	Retrains with a new seed	0.937 and 0.946 again
A PET scan is never newer than its visit	Compares dates	0 violations

The two mistakes it found

- **Education from the future.** For about 0.1% of people, a missing education value at an early visit had been filled in from a later visit.
- **Scans after the visit.** Many PET scans had been matched to a visit that took place shortly before the scan.

Both use information that would not have been available at the time. We fixed both and re-ran every model. The main results changed only in the third decimal place.

A second set of checks: does the screen behave sensibly?

`check_clinical_behaviour.py` runs 28 checks on the saved model: the screen asks for exactly the trained features; a worse rating or test score never lowers the risk; contradictory entries are flagged. 27 pass. The one that does not: for people under 65 with MCI the model says 35% where 27% actually converted. We report it openly.

Say it like this: "We wrote a program that checks our own work in seventeen ways. It found two places where we had accidentally used information from the future. We fixed both and re-ran everything."

16 The doctor's screen

In one line: A web page where a clinician enters what they know and sees the 3-year risk, what happened to similar people, and what moved the estimate.

Part	File	Job
The page	`app/web/` (React)	The form and the results
The engine	`app/api.py`	Receives the entries and returns the result
The translator	`scripts/inference.py`	Turns what was typed into the 48 features (for example a MoCA score into a z-score), runs the model, runs SHAP
The model	`models/`	The modality-dropout model from chapter 12

What the clinician does

- Fills in section 1 (age, sex, education). Everything else is optional.
- Marks each other section as available or not: heart and blood vessels, clinician's assessment, test scores, genes, brain scan.
- Leaves unknown boxes empty. Empty means 'not available'; nothing is filled in with a made-up value.

What the page shows

- The estimated 3-year risk.
- Next to it, what actually happened to research participants of the same age group and status.
- Which information was used and which was missing.
- Bars showing which kind of information pushed the risk up or down.
- A red warning when entries contradict each other, for example a test score of 0 with status 'normal'.

The page asks for exactly the items the model was trained on and nothing else. It also contains a box for asking the evidence chatbot.

To run it

```
python -m uvicorn app.api:app --port 8000
# then open http://localhost:8000 in a browser
```

Example: Leave the defaults (age 72, status Normal, CDR 0): the risk is about 1%. Change status to MCI, CDR to 1.5 and MoCA to 21: it rises to about 30%. Change status back to Normal with MoCA 0: a red box appears, because those entries contradict each other.

Careful: It does not recommend medicines or treatment. Our data only record what happened to people; they cannot tell us what would have happened with a different treatment.

Say it like this: "We built a simple screen for a clinician. It gives the estimated risk, shows what drove it, and warns when the entries do not make sense together."

17 Limits, the presentation, and questions

What the project cannot claim

- It was tested on past research data, not on patients in a real clinic.
- The volunteers are mostly White and highly educated, with an average age of about 71. Results may differ for other people.
- About half of the eligible people were not followed for 3 years and had to be left out.
- We cannot see people's health in middle age, when cardiovascular problems matter most for the brain.
- Very few cognitively normal people developed dementia (198 of 12,377), so results for that group are uncertain.
- The model leans on the doctor's rating. It is best described as estimating progression given the current stage, not early detection in people with no signs.
- Scanned people have short follow-up, and the chatbot's library is small.

Everything we found, in one table

Question	Answer
Can dementia within 3 years be predicted?	0.94 for everyone; 0.84 within MCI; 0.88 within normal (few cases)
Does it work at hospitals it never saw?	Yes, about 0.95 at 9 unseen centres
Is advanced AI better than simple models?	No, all five score the same
Does cardiovascular information help?	Little extra value in this group and setup: about +0.001
Does history help?	Only slightly: +0.000 to +0.005
Do PET numbers help?	Not shown
Did the image pipeline work?	Yes: 0.98 agreement with the experts on 318 scans
Is the image model better than one simple number?	No: 0.96 against 0.98
Can it cope with missing information?	Yes, after training with hidden sections
Did our checks pass?	17 of 17, after fixing two mistakes they found

A slide order that works (about 10 slides)

Slide	Content	Chapter
1	The question in one sentence, and 'research prototype'	1
2	The data: 57,038 people, five kinds of information	3
3	Who we studied and the fair test (use the split picture)	6
4	Five models, one table of scores	7
5	0.94 / 0.84 / 0.88: reading the score honestly	7
6	What matters: thinking today, not cardiovascular or history	8, 9

Slide	Content	Chapter
7	PET numbers and the image experiment	10, 13
8	How we checked ourselves: 17 checks, two mistakes fixed	15
9	Live demo: the screen and one chatbot question	14, 16
10	Limits and what we do not claim	17

Questions you will probably get

Question	Short answer
Is 0.94 too good to be true?	It mixes two groups with very different risk (41.5% against 1.6%). Status alone already gives 0.84. Inside MCI the model scores 0.84, the fairer number.
Isn't using the doctor's rating cheating?	No. It is recorded at the first visit and the outcome comes later. But the model mostly measures how far along someone already is, and we say so.
Why 3 years?	Long enough to be useful, short enough that enough people were followed that long. We also computed 2 years.
Why did cardiovascular not help?	People join at about 71; the damage happens in middle age, which we cannot see, and its effect is probably already in the thinking tests. We do not claim it is unimportant.
How do you know the model did not see the answers?	Nobody is in two groups; 9 whole hospitals were never seen; with shuffled answers the score falls to 0.51; features were checked to use nothing from later visits.
Why split by person and not by visit?	A person has several visits. If they were on both sides, the model would recognise the person instead of learning the pattern.
What is a z-score and why use it?	A score compared with healthy people of the same age, sex and education. It makes old and new tests, and different people, comparable.
Why XGBoost as the main model?	It scored as well as any other, handles missing values by itself, trains fast, and can be explained exactly with SHAP.
Why not predict from the scan images?	Only 5 of 446 people with images developed dementia. Nothing can learn from 5 examples.
What did your checks find?	Education filled in from a later visit, and scans dated after the visit. Both fixed; results moved only in the third decimal.
Is it fair to different groups?	Scores are similar across sex, race and education. It is weaker for people aged 85 and over.
Why is it called CardioCog-RAG?	Cardio: the cardiovascular question. Cog: cognitive data. RAG: the design of the evidence chatbot.
Can doctors use it now?	No. It is a research prototype tested only on past research data.
What would you do next?	Test on a different data set, use a longer window, and study cognitively normal people separately, where cardiovascular information and PET have more room to matter.

Do not say

- 'It diagnoses dementia.' It estimates a risk.
- 'Cardiovascular health is useless.' Say it added little in this group and setup.
- 'PET does not help.' Say we could not show that it helps.

- '0.94' on its own. Always give 0.84 for MCI with it.
- 'It was tested on patients.' It was tested on past research volunteers.

If you do not know an answer, say: 'I will check that in our results', and look in `docs/claims_verification.md`, which traces every number to its source.